ICT 2121 Data Structures

Midterm Examination-September, 2023

Scheme of Valuation

Date: 25/9/2023 Time: 2 hrs

Type: MCQ

Q1. The output of the following error-free code is _____ (0.5)

```
int *xyz(int *b)
{
   int a[5] = {1,2,3,4,5};
   return(a);
}
int main ()
{
     int a[3] = {10,20,30}, *p, i;
     p=xyz(a);
     printf("%d ", p [2]);
     return 0;
  }
```

1. 3
2. **Garbage /undefined value
3. 30
4. 2

Q2. In a multiple stack implementation, what should be done to prevent one stack from overrunning into the other? (0.5)

1. ** Define a fixed boundary between the stacks
2. Implement stack balancing algorithms
3. Use a larger memory space.
4. Use separate memory spaces for each stack

Q3. Output of the following error-free code snippet is ___. (0.5)

Assume the size of character datatype is 1 byte.

```
char s [10][20];
printf("%d %d", s, (s+1));
```

1. **6356516, 6356536
2. 6356516, 6356526
3. 6356516, 6356517

4. None of the above

Q4. Consider the following variable declarations: (0.5)

int *a; float *x; char *p;

Output of the following statement is:

printf("%d, %d, %d", sizeof(a), sizeof(x), sizeof(p));

1. 2, 4, 1
2. **4, 4, 4
3. 4,2,1
4. 4, 8, 2

Q5. Suppose multiple stacks are implemented by dividing MAX_SZ into N equal (except for the last stack) segments. Which of the following C statements could be used to check if the ith stack is empty ? (0.5)

1. **If top[i] == boundary[i])
2. If top[i] == boundary[i+1])
3. If top[i+1] == boundary[i])
4. None of the above

Q6. How many stacks are required to implement a queue where no other data structure like arrays or linked lists is available? (0.5)

1. **2
2. 3
3. 4
4. 1

Q7. Suppose you are given an implementation of a queue of integers. The operations that can be performed on the queue are:
i. isEmpty (Q) — returns true if the queue is empty, false otherwise.
ii. delete (Q) — deletes the element at the front of the queue and returns its value.
iii. insert (Q, i) — inserts the integer i at the rear of the queue.
Consider the following function:

```
void f (queue Q)
{
  int i ;
  if (!isEmpty(Q)) {
    i = delete(Q);
    f(Q);
    insert (Q, i);
  }
}
```

What operation is performed by the above function f? (0.5)

1. Leaves the queue Q unchanged
2. **Reverse the order of the elements in the Queue Q
3. Deletes the element at the front of the Queue Q and inserts it at the rear keeping the other elements in the same order

4. Empties the Queue Q

Q8. What is the purpose of the "rowTerms" array in the fast transpose of a sparse matrix? (0.5)

1. To store matrix dimensions of the transpose matrix
2. ** To store the number of non-zero elements in each row of the transpose matrix
3. To store the row indices of non-zero elements of the transpose matrix
4. To store the column indices of non-zero elements of the transpose matrix

Q9. The reverse () function is supposed to reverse a singly linked list. One line is missing at the end of the function.

```
struct node
{ int data; struct node* next; } ;
/* head_refer is a pointer that points to the head (or start) pointer of linked list * /
struct node* reverse( struct node *head_refer )
{
 struct node* prev  = NULL;
 struct node* current = head_refer;
 struct node* next1;
 while (current != NULL)
 {
   next1 = current->next;
  current->next = prev;
  prev = current;
  current = next1;
 }
 /*ADD A STATEMENT HERE*/

  return(head_refer);
  }
```

What should be the statement so that the function correctly reverses a linked list? (0.5)

1. **head_refer = prev;
2. head_refer = current;
3. head_refer = next;
4. head_refer = NULL;

Q10. Consider the following function:

```
void test (struct Node* head)
{
   int count = 0;
   while (head->next!=NULL) {
      if (count%2 == 0)
         printf(" %d ", head->data);
      count++;
      head = head->next;
```

```
   }
printf(" %d ", count); //REFER THIS LINE
}
```

if list contains 1->2->3->4->5->6->NULL, what would be the value of count printed after while loop?

(0.5)

1. 2
2. 3
3. **5
4. 6

Q11. Design a user defined function to sort an array of strings in descending order using bubble sort technique without using any string library function. (3)

Answer

```
    void stringsort(char str[][], int n){
    int i, j;
    char temp[20];
    void strcopy(char[], char[]);
     int strcomp(char[], char[]);
     int strlength(char []);
    for(i=0;i<n-1;i++)
for(j=0;j<n-i-1;j++)
  if(strcomp(str[j],str[j+1])>0)
  {
          strcopy(temp,str[j]);
          strcopy(str[j],str[j+1]);
          strcopy(str[j+1],temp);
  }
   printf("The strings appears after sorting :\n");
   for(i=0;i<=n;i++)
printf("%s\n",str[i]);

}
int strlength(char s[])
{ int count=0, i=0;
  while(s[i]!='\0')
     count++;
  return count;
}
void strcopy(char s1[], char s2[])
{
int i;
for(i=0;i<strlength(s2);i++)
   s1[i]=s2[i];
s1[i]='\0';
}
```

```
int strcomp(char s1[], char s2[])
{
  int i=0;
   while(s1[i]!='\0' &&s2[i]!='\0')
   {
     if(s1[i]<s2[i])
       return -1;
     else if(s1[i]>s2[i])
       return 1;
     else{
       i++;
       continue;}
    if(s1[i]=='\0' && s2[i]=='\0')
          return 0;
    else if(s1[i]=='\0' && s2[i]!='\0')
          return -1;
    else if(s1[i]!='\0' && s2[i]=='\0')
          return 1;
  }
}
```

stringsort()- 2 marks, strlength() and strcopy()-0.5 marks together, strcomp()-0.5 marks

Q12. Develop a C program to read and display employee details according to the following requirements:
a. Define a structure employee with data members to store an employee id, name work experience (in years), and gender.
b. Write a user defined function to read an employee details. The function prototype is:s truct employee_read();
c. Write user defined function to display the details of the senior most and junior most employee details. The function prototype is: void display(struct employee[])

Answer:

Structure defn:0.5 marks, read fn: 1 mark, display fn : 1 marks, main fn:05 marks

```
#include<stdio.h>
int n;
typedef struct {
   int years;
   int id;
   char name[20];
   char gender;}employee;
employee read()
{
    employee e;
    scanf("%d %d %s %c", &e.years, &e.id, &e.name, &e. gender);
    return(e);
}
void disp (employee e[])
{
```

```
    int i; employee large=e[0], small=e[0];
    for(i=1;i<n;i++)
    {

       if(e[i].years>large.years) large=e[i];
       if(e[i].years<small.years) small=e[i];

    }
      printf("Senior employee :%d %d %s %c\n ", large.years, large.id, large.name, large.gender);
      printf("Junior employee :%d %d %s %c ", small.years, small.id, large.name, small.gender);
}


int main()
{
    employee e[10];int i;
    printf("Number of employee:");
    scanf("%d", &n);
    for (i=0;i<n;i++)
        e[i]=read();
   disp(e);
}
```

Q13. Write the steps to convert a given infix expression into its corresponding prefix form. Convert the following infix expression into its prefix form and show the stack content at each step:

(A + B * (C - D)/E). (3)

Answer: [1.5M for proper steps; 1.5m for the proper conversion]

Steps involved to convert a given infix expression into its corresponding prefix form are as given below:

- Reverse the expression.
- Read the expression from left to right and
    - If an operand if scanned, copy it to the output (left to right)
    - If a right parenthesis is scanned push it into stack
    - When a left parenthesis is encountered, the operator at the top of the stack is popped off the stack and copied to the output until the symbol at the top of the stack is the right parenthesis. When that occurs, both the parentheses are discarded.
    - If an operator is scanned and has higher or equal precedence than the operator at the top of the stack, the operator being scanned is pushed onto stack.
    - While the precedence of the operator being scanned is lower than to te precedence of the operator at the top of the stack, the operator at the top of stack is popped and copied into output.
    - When the end of the expression is reached on the input scan, the remaining operators in the stack are popped and copied to the output.
- Reverse the output.

To convert the expression: (A + B * (C - D) / E)

Reverse the expression:) E /) D - C (* B + A (

Scanned Element	Stack Content	Output	Description
)	)		Push(')')
E	)	E	Place in output (E)
/	) /	E	Push (/)
)	) /)	E	Push ( ))
D	) /)	ED	Place in output (D)
-	) /) -	ED	Push (-)
C	) /) -	EDC	Place in output(C)
(	) /	EDC-	Place in output (Pop (-)), Pop ( ))
*	) / *	EDC-	Push(*)
B	) / *	EDC-B	Place in output(B)
+	)+	EDC-B	Place in output (Pop(/), Pop(*)), Push(+)
A	)+	EDC-B*/A	Place in output(A)
(		EDC-B*/A+	Place in output(Pop(+)), Pop())

Reverse the expression: **+A/*B-CDE**

Q14. Describe how to access the element a[i][j] of a 2-dimensional matrix using pointers. Write a function to display elements of a 2-dimensional matrix using pointers. (2)

Answer

```
void display(int a[10][10], int m, int n)
{
int i, j;
for(i=0;i<m;i++)
  for(j=0;j<n;j++)
    printf("%d", *(*(a+i)+j));
return;
}
```

Formula *(*(a+i)+j)-0.5 marks, function display()-1.5 marks

Q15. A music player uses a circular queue to manage the songs in a playlist. The songs are added to the queue in the order they are selected by the user (**Figure Q.11**). Initially the front and rear=0. The music player has the following features:

- Play: Plays the song at the front of the queue and removes it from the queue.
- Next: Skips the current song and plays the next song in the queue. The skipped song is removed from the queue.
- Repeat: Repeats the current song and adds it to the rear of the queue.

Suppose the playlist has the following songs in the queue:

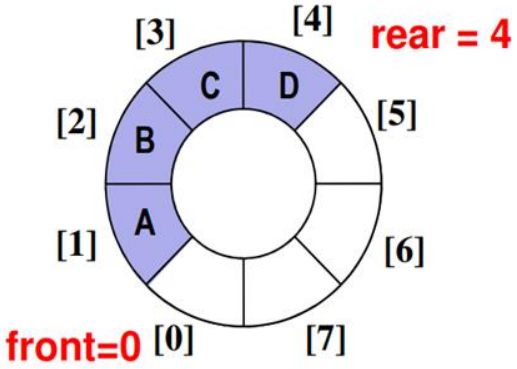


Figure Q.11: Song Representation

a. Apply the features and represent the state of the queue after the following sequence of operations? [Play, Next, Repeat, Next]
b. Write the snippet for playing a song from the list. (4)

Answer

A: The state of the queue after the sequence of operations will be:

- **Play**: Plays song A and removes it from the queue.

Front	**B**	**C**	**D**	**Rear**

- **Next**: Skips song B and plays song C. Song B is removed from the queue.

Front	**C**	**D**	**Rear**

- **Repeat**: Repeats song C and adds it to the rear of the queue.

Front	**C**	**D**	**C**	**Rear**

- **Next**: Skips song C and plays song D. Song C is removed from the queue.

Front	**D**	**C**	**Rear**

Final state: D C _ _ _ _ _ _ (front = 3, rear = 5)

Each state representation carries **0.5M**

B: Snippet – **2M**

```
void play()
{
      if (front == -1 && rear == -1)
{
      printf("No Songs to Play");
   }
else if (front == rear)
```

```
{
        printf("%s has been played\n", queue[front]);
        front = rear = -1;
    }

else {
        printf("%s has been played\n", queue[front]);
        front = (front + 1) % SIZE;
    }
```

Q16. Write a complete C program to perform the following operations by displaying appropriate messages:

a. Read a binary number from user

b. Store each bit of binary number as a node in Singly Linked List

c. Convert stored number in list to decimal by traversing through list.

[Example: - if user entered 10110 as input, SLL (Singly Linked List) content would be 1->0->1->1->0->NULL and decimal value would be 22] (4)

Answer [struct: 0.5m, reading binary number from user: 0.5m, placing digits in list: 1.5m, conversion by traversing list: 1.5m]

```
struct Node
{ int data;
struct Node* next;
};
struct Node *head=NULL;

int power(int x,int y)
{   int a=1;
   while(y>0){
a=a*x;
y--;
  }
return(a);
}
int decimalValue(struct Node *head,int ndigits)
{          int res = 0,n=ndigits;
while (head != NULL)
{
if(head->data>0) res=res+power(2,(n-1));
head = head->next;
n=n-1;
}
return res;
}
struct Node *insFirst(struct Node *h,int bit)
{          struct Node *t=(struct Node*)malloc(sizeof(struct Node));
t->data=bit;
```

```
t->next=NULL;
if(h==NULL) return(t);
else
{       t->next=h;
h=t;
return(h);
}

}
int main()
{       int binary,b; //b is bit
printf("Enter a binary number: "); scanf("%d",&binary);
int t=binary,nd=0; //nd is number of digits
while(t>0)
{       b=t%10;
head=insFirst(head,b);
t=t/10;
nd++;
}
printf("Decimal value is %d   %d",nd,decimalValue(head,nd));
return 0;
}
```

Q17. Illustrate a process to check for duplicate parenthesis using stack by showing its content at each step.

Example: **Input:** (a **+** ((x + y)) + z / p + s)
Output: True. Expression has duplicates.
Input: ((x + y) *z)
Output: False. Expression does not have duplicates. (3)

Answer: [1.5M for the proper explanation; 1.5M for the illustration of the example – both valid and invalid cases]

- Iterate through the given expression from left to right.
- For each character in the expression, if the character is an open parenthesis '(' or any of the operators or operands, push it to the top of the stack.
- If the character is close parenthesis ')', then pop characters from the stack till matching open parenthesis '(' is found and a counter is used, whose value is incremented for every character encountered till the opening parenthesis '(' is found.
- If the number of characters encountered between the opening and closing parenthesis pair, which is equal to the value of the counter, is less than 1, then a pair of duplicate parentheses is found else there is no occurrence of redundant parenthesis pairs.

Example: a+ ((x+y))

Scanned Element	Stack	Action
a	a	Push(a)
+	a+	Push (+)
(	a+ (	Push (()
(	a+ ((	Push (()

x	a+ ((x	Push(x)
+	a+ ((x+	Push (+)
y	a+ ((x+y	Push(y)
)		Start popping element one by one until you reach the corresponding '('. Every pop () increases the counter value by 1. When '(' is encountered as popped element, check the counter, if it is less than 1, then there is duplicates otherwise not.
)		Start popping element one by one until you reach the corresponding '('. Every pop () increases the counter value by 1. When '(' is encountered as popped element, check the counter, if it is less than 1, then there is duplicates otherwise not.

Q18. Write the table representation of the below sparse matrix. Demonstrate the fast transpose algorithm by showcasing the outcomes at each iteration. Additionally, display the contents of the *rowTerms* and *startingPosition* arrays. (3)

[0 0 20 0 40

0 0 50 70 0

0 0 0 0 0

0 20 60 0 0]

Answer

[Table representation: 0.5 mark, rowTerms: 0.5 mark, startingPosition: 0.5 mark, correct writing of startingPosition and Transposed matrix: 1.5 mark, 0 marks if only code is written or directly the transposed sparse representation is written]

[0 0 20 0 40

0 0 50 70 0

0 0 0 0 0

0	<4,5,6> (row, col, NZeroTerms)
1	<0,2,20>
2	<0,4,40>
3	<1,2,50>
4	<1,3,70>
5	<3,1,20>

6	<3,2,60>

0 20 60 0 0]

0	1	3	1	1
0	1	2	3	4

RowTerms:

1	1	2	5	6
0	1	2	3	4

startingPosition:

Iteration Count	startingPosition	Transposed Sparse Representation
1	1 1 **3** 5 6 0 1 2 3 4	0 <5,4,6> 1 2 **<2,0,20>** 3 4 5 6
2	1 1 3 5 **7** 0 1 2 3 4	0 <5,4,6> 1 2 <2,0,20> 3 4 5 6 **<4,0,40>**

3

1	2	**4**	5	7
0	1	2	3	4

0	<5,4,6>
1	
2	<2,0,20>
3	**<2,1,50>**
4	
5	
6	<4,0,40>

4

1	1	4	**6**	7
0	1	2	3	4

0	<5,4,6>
1	
2	<2,0,20>
3	<2,1,50>
4	
5	**<3,1,70>**
6	<4,0,40>

5

1	**2**	4	6	7
0	1	2	3	4

0	<5,4,6>
1	**<1,3,20>**
2	<2,0,20>
3	<2,1,50>
4	
5	<3,1,70>
6	<4,0,40>

6

1	2	**5**	6	7
0	1	2	3	4

0	<5,4,6>
1	<1,3,20>
2	<2,0,20>
3	<2,1,50>
4	**<2,3,60>**
5	<3,1,70>
6	<4,0,40>